

Bateria de testes — Trabalho 1 (Simplex Duas Fases)

Pesquisa Operacional — 2026/2 — Ciência da Computação — IFNMG Montes Claros

Esta bateria tem **107 casos**, organizados em sete grupos de complexidade crescente. Todos os gabaritos em `esperado/` foram produzidos por uma implementação de referência e conferidos por uma segunda implementação independente; os casos do grupo G têm, além disso, o valor ótimo certificado matematicamente por construção.

Como usar esta bateria: resolva os grupos **na ordem**. Um caso do grupo E só faz sentido depois que todo o grupo A passa. Rode `python3 verificar.py ./simplex --grupo A` e só avance quando o grupo inteiro estiver verde.

Índice

- [Grupo A](#) — 11 casos
- [Grupo B](#) — 7 casos
- [Grupo C](#) — 6 casos
- [Grupo D](#) — 7 casos
- [Grupo E](#) — 22 casos
- [Grupo F](#) — 41 casos
- [Grupo G](#) — 13 casos
- [Matriz de cobertura](#)
- [Política de comparação](#)

Matriz de cobertura

Cada célula indica o caso do grupo F que exercita aquela combinação. Os grupos A-E cobrem as mesmas facetas isoladamente e em contextos adicionais.

tipos de restrição \ propriedade	ótima única	múltiplas	degenerada	degen. + múltiplas	inviável	ilimitada
só $\leq$	f11	f12	f13	f14	— (impossível)	f16
só $\geq$	f21	f22	f23	f24	f25	f26
só $=$	f31	f32	f33	f34	f35	f36
$\leq$ e $\geq$	f41	f42	f43	f44	f45	f46
$\leq$ e $=$	f51	f52	f53	f54	f55	f56
$\geq$ e $=$	f61	f62	f63	f64	f65	f66
$\leq$, $\geq$ e $=$	f71	f72	f73	f74	f75	f76

Por que f15 não existe: com **apenas** restrições $\leq$ e todos os $b \geq 0$, a origem $x = 0$ é sempre viável — logo o problema **nunca** pode ser inviável. É a única célula vazia da matriz, e o motivo é teórico, não uma omissão. (Se algum b for negativo, a restrição $\leq$ vira $\geq$ na normalização e a combinação deixa de ser "somente $\leq$ " — veja e22 .)

Facetas adicionais cobertas fora da grade: $b = 0$ (a06 , b05), $b < 0$ (b07 , d04 , e07 , e21 , e22), restrição redundante (d03), minimização (11 casos), coeficientes fracionários (a11 , b01), variável ausente das restrições (e02), objetivo com coeficiente zero (e14), ALTERNATIVA: RAI0 (e19), Fase 2 sem nenhuma iteração (b02 , b04 , d01 , d02) e Fase 1 sem nenhuma iteração (todo o grupo A).

Grupo A — Fundamentos: somente restrições $\leq$ com $b \geq 0$

Nenhuma variável artificial é criada: a Fase 1 é dispensada e o contador de iterações da Fase 1 é 0 . Se algum caso deste grupo falhar, o problema está na leitura da entrada, na montagem do tableau ou no pivoteamento — não no método das Duas Fases.

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
a01	1	1	<=	MAX	OTIMA	12	NAO	NAO	0 1	O menor PL possível: confira o tableau inteiro a mão.
a02	2	1	<=	MAX	OTIMA	20	NAO	NAO	0 1	Só a variável mais lucrativa entra na base.
a03	2	2	<=	MAX	OTIMA	12	NAO	NAO	0 1	Duas variáveis, duas restrições
a04	3	3	<=	MAX	OTIMA	13	NAO	NAO	0 2	Exemplo clássico do livro-texto; duas iterações na Fase 2.
a05	2	2	<=	MAX	OTIMA	9	NAO	NAO	0 1	A variável com lucro negativo nunca entra na base.
a06	2	2	<=	MAX	OTIMA	15	NAO	NAO	0 1	A folga da segunda restrição já começa básica valendo zero.
a07	4	2	<=	MAX	OTIMA	27.6	NAO	NAO	0 2	Mais variáveis que restrições
a08	2	5	<=	MAX	OTIMA	28	NAO	NAO	0 2	Mais restrições que variáveis (algumas inativas)
a09	2	1	<=	MIN	OTIMA	0	NAO	NAO	0 0	Com custos positivos e somente restrições <=, o ótimo consiste em não produzir nada.
a10	2	2	<=	MIN	OTIMA	-13	NAO	NAO	0 2	Minimizar custos negativos equivale a maximizar; confira o sinal de Z.
a11	2	2	<=	MAX	OTIMA	10.8571	NAO	NAO	0 2	Verifique se sua leitura aceita números com ponto decimal.

Grupo B — Uma única restrição >=

Aqui aparece a primeira variável artificial e, com ela, a Fase 1. São os menores casos possíveis em que o método das Duas Fases realmente roda.

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
b01	2	3	<= >=	MAX	OTIMA	3675	NAO	NAO	1 2	Mesmo problema resolvido no quadro pelo Big-M.
b02	1	1	>=	MIN	OTIMA	6	NAO	NAO	1 0	A Fase 2 termina sem nenhuma iteração: a base da Fase 1 já é ótima.
b03	2	2	<= >=	MAX	OTIMA	40	NAO	NAO	1 1	A artificial sai na Fase 1 e o excesso fica positivo no ótimo.
b04	2	2	<= >=	MIN	OTIMA	18	NAO	NAO	1 0	Restrição >= ativa no ótimo
b05	2	2	<= >=	MAX	OTIMA	24	NAO	NAO	1 2	Artificial básica valendo zero desde o início da Fase 1.
b06	2	2	<= >=	MIN	OTIMA	160	NAO	NAO	1 0	Dieta mínima (2 nutrientes)
b07	2	2	<= >=	MAX	OTIMA	11	NAO	NAO	0 2	Depois da normalização não sobra nenhuma artificial: Fase 1 dispensada.

Grupo C — Várias restrições >=

Duas ou mais artificiais simultâneas. A Fase 1 passa a precisar de várias iterações, e a linha Z da Fase 1 é a soma de várias linhas do tableau.

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
c01	2	3	>=	MIN	OTIMA	2.8	NAO	NAO	3 1	Três artificiais; a Fase 1 precisa de varias iterações.
c02	2	3	>=	MIN	OTIMA	34	NAO	NAO	3 1	Somente restrições >=
c03	2	4	<= >=	MAX	OTIMA	540	NAO	NAO	2 2	Duas >= e duas <=
c04	2	4	<= >=	MAX	OTIMA	290	NAO	NAO	2 2	Sucos Vitamix (duas >= simultâneas)
c05	3	4	<= >=	MIN	OTIMA	150	NAO	NAO	3 2	Três variáveis com três pisos
c06	3	5	>=	MIN	OTIMA	39.5	NAO	NAO	5 1	Cinco restrições >=

Grupo D — Restrições de igualdade =

Igualdades criam artificial sem criar excesso. Inclui o caso da restrição redundante, em que sobra artificial básica em zero ao fim da Fase 1 (Seção 3.4 do enunciado).

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
d01	2	2	$\leq$ =	MAX	OTIMA	13	NAO	NAO	2 0	Uma igualdade
d02	2	2	=	MAX	OTIMA	10	NAO	NAO	2 0	O sistema já determina o único ponto viável: a Fase 2 não itera.
d03	2	2	=	MAX	OTIMA	12	NAO	NAO	1 1	Ao fim da Fase 1 sobra artificial básica em zero com a linha nula: remova a linha.
d04	2	2	$\leq$ =	MAX	OTIMA	7	NAO	NAO	1 1	Igualdade com b negativo
d05	2	3	$\leq$ = $\geq$	MIN	OTIMA	3.4	NAO	NAO	2 1	Exemplo clássico de Duas Fases dos livros-texto.
d06	3	3	$\leq$ =	MAX	OTIMA	55	NAO	SIM	2 0	Duas igualdades em três variáveis: sobra uma variável básica em zero.
d07	3	2	=	MAX	OTIMA	18	NAO	SIM	2 0	A terceira variável e forçada a zero: base degenerada.

Grupo E — Cada propriedade especial isolada

Ilimitada, inviável, múltiplas soluções, degenerescência, **b** negativo e aresta ótima ilimitada, cada uma no menor exemplo possível, para você localizar exatamente qual detecção está falhando.

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
e01	2	2	$\leq$	MAX	ILIMITADA	—	—	—	—	Nenhuma linha tem coeficiente positivo na coluna que entra.
e02	2	1	$\leq$	MAX	ILIMITADA	—	—	—	—	x2 não aparece em nenhuma restrição.
e03	2	1	$\geq$	MAX	ILIMITADA	—	—	—	—	A Fase 1 termina normalmente; a ilimitacao só aparece na Fase 2.
e04	2	1	$=$	MAX	ILIMITADA	—	—	—	—	Ilimitada com igualdade
e05	2	2	$\leq$ $\geq$	MAX	INVIÁVEL	—	—	—	—	A Fase 1 termina com $w > 0$.
e06	2	2	$=$	MAX	INVIÁVEL	—	—	—	—	Inviável: duas igualdades contraditórias
e07	2	2	$\leq$ $=$	MAX	INVIÁVEL	—	—	—	—	Depois da normalização a igualdade exige $x_1 = -3$, impossível com $x \geq 0$.
e08	2	2	$\geq$	MAX	INVIÁVEL	—	—	—	—	Inviável somente com $\geq$
e09	2	3	$\leq$ $\geq$	MIN	INVIÁVEL	—	—	—	—	Inviável com três restrições $\geq/\leq$
e10	2	3	$\leq$	MAX	OTIMA	20	SIM	NAO	0 2	Os dois extremos da aresta ótima da aula 5.
e11	2	2	$\leq$ $\geq$	MAX	OTIMA	20	SIM	NAO	1 1	Múltiplas soluções com restrição $\geq$
e12	2	2	$\leq$ $=$	MAX	OTIMA	4	SIM	NAO	2 0	Múltiplas soluções com igualdade
e13	2	2	$\leq$ $\geq$	MIN	OTIMA	4	SIM	NAO	2 0	Múltiplas soluções em minimizacao
e14	2	2	$\leq$	MAX	OTIMA	12	SIM	NAO	0 1	Custo reduzido nulo em coluna não básica, mesmo sem aresta "inclinada".
e15	2	2	$\leq$	MAX	OTIMA	18	NAO	SIM	0 2	Exemplo do Bloco 9 da aula 5.
e16	2	2	$\geq$	MIN	OTIMA	8	NAO	SIM	2 0	As duas restrições se cruzam no vértice ótimo.
e17	2	3	$\leq$	MAX	OTIMA	4	SIM	SIM	0 2	Três retas passam pelo vértice ótimo: base degenerada E custo reduzido nulo. Compare a ALTERNATIVA com a solução ótima - são o mesmo ponto!

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
e18	2	2	<= >=	MAX	OTIMA	5	SIM	SIM	1 1	As duas restrições descrevem a mesma reta.
e19	2	1	<=	MAX	OTIMA	1	SIM	NAO	0 1	Custo reduzido nulo, mas sem teste da razão válido: o conjunto ótimo é uma semirreta.
e20	2	1	<=	MAX	OTIMA	0	NAO	NAO	0 0	Ótimo na origem (todos os lucros negativos)
e21	2	2	<=	MAX	OTIMA	18	NAO	NAO	1 1	A primeira restrição vira >= e cria uma artificial.
e22	2	1	<=	MAX	INVIAVEL	—	—	—	—	$x_1 + x_2 \leq -3$ com $x \geq 0$ não tem solução.

Grupo F — Grade completa: tipo de restrição × propriedade

Cada célula da matriz abaixo combina um conjunto de tipos de restrição com uma propriedade da solução. É a parte da bateria que garante que nenhuma combinação ficou sem teste.

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
f11	2	2	$\leq$	MAX	OTIMA	12	NAO	NAO	0 1	Tipo [só $\leq$] x propriedade [ótima única]
f12	2	3	$\leq$	MAX	OTIMA	20	SIM	NAO	0 2	Tipo [só $\leq$] x propriedade [múltiplas]
f13	2	2	$\leq$	MAX	OTIMA	18	NAO	SIM	0 2	Tipo [só $\leq$] x propriedade [degenerada]
f14	2	2	$\leq$	MAX	OTIMA	4	SIM	SIM	0 1	Tipo [só $\leq$] x propriedade [degenerada+múltiplas]
f16	2	2	$\leq$	MAX	ILIMITADA	—	—	—	—	Tipo [só $\leq$] x propriedade [ilimitada]
f21	2	2	$\geq$	MIN	OTIMA	8	NAO	NAO	2 1	Tipo [só $\geq$] x propriedade [ótima única]
f22	2	2	$\geq$	MIN	OTIMA	4	SIM	NAO	2 0	Tipo [só $\geq$] x propriedade [múltiplas]
f23	2	2	$\geq$	MIN	OTIMA	8	NAO	SIM	2 0	Tipo [só $\geq$] x propriedade [degenerada]
f24	2	2	$\geq$	MIN	OTIMA	4	SIM	SIM	2 0	Tipo [só $\geq$] x propriedade [degenerada+múltiplas]
f25	2	2	$\geq$	MIN	INVIAVEL	—	—	—	—	Tipo [só $\geq$] x propriedade [inviável]
f26	2	2	$\geq$	MAX	ILIMITADA	—	—	—	—	Tipo [só $\geq$] x propriedade [ilimitada]
f31	2	2	$=$	MAX	OTIMA	10	NAO	NAO	2 0	Tipo [só $=$] x propriedade [ótima única]
f32	2	1	$=$	MAX	OTIMA	4	SIM	NAO	1 0	Tipo [só $=$] x propriedade [múltiplas]
f33	2	2	$=$	MAX	OTIMA	12	NAO	SIM	2 0	Tipo [só $=$] x propriedade [degenerada]
f34	3	2	$=$	MAX	OTIMA	4	SIM	SIM	2 0	Tipo [só $=$] x propriedade [degenerada+múltiplas]
f35	2	2	$=$	MAX	INVIAVEL	—	—	—	—	Tipo [só $=$] x propriedade [inviável]
f36	2	1	$=$	MAX	ILIMITADA	—	—	—	—	Tipo [só $=$] x propriedade [ilimitada]
f41	2	3	$\leq$ $\geq$	MAX	OTIMA	3675	NAO	NAO	1 2	Tipo [$\leq$ e $\geq$] x propriedade [ótima única]
f42	2	2	$\leq$ $\geq$	MAX	OTIMA	20	SIM	NAO	1 1	Tipo [$\leq$ e $\geq$] x propriedade [múltiplas]

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
f43	2	3	$\leq$ $\geq$	MAX	OTIMA	18	NAO	SIM	1 2	Tipo [$\leq$ e $\geq$] x propriedade [degenerada]
f44	2	2	$\leq$ $\geq$	MAX	OTIMA	4	SIM	SIM	1 1	Tipo [$\leq$ e $\geq$] x propriedade [degenerada+múltiplas]
f45	2	2	$\leq$ $\geq$	MAX	INVIABEL	—	—	—	—	Tipo [$\leq$ e $\geq$] x propriedade [inviável]
f46	2	2	$\leq$ $\geq$	MAX	ILIMITADA	—	—	—	—	Tipo [$\leq$ e $\geq$] x propriedade [ilimitada]
f51	2	2	$\leq$ =	MAX	OTIMA	12	NAO	NAO	2 1	Tipo [$\leq$ e =] x propriedade [ótima única]
f52	2	2	$\leq$ =	MAX	OTIMA	4	SIM	NAO	2 0	Tipo [$\leq$ e =] x propriedade [múltiplas]
f53	2	2	$\leq$ =	MAX	OTIMA	8	NAO	SIM	1 0	Tipo [$\leq$ e =] x propriedade [degenerada]
f54	3	2	$\leq$ =	MAX	OTIMA	4	SIM	SIM	1 0	Tipo [$\leq$ e =] x propriedade [degenerada+múltiplas]
f55	2	2	$\leq$ =	MAX	INVIABEL	—	—	—	—	Tipo [$\leq$ e =] x propriedade [inviável]
f56	2	2	$\leq$ =	MAX	ILIMITADA	—	—	—	—	Tipo [$\leq$ e =] x propriedade [ilimitada]
f61	2	2	= $\geq$	MIN	OTIMA	3	NAO	NAO	2 1	Tipo [$\geq$ e =] x propriedade [ótima única]
f62	3	2	= $\geq$	MIN	OTIMA	4	SIM	NAO	2 0	Tipo [$\geq$ e =] x propriedade [múltiplas]
f63	2	2	= $\geq$	MIN	OTIMA	8	NAO	SIM	2 0	Tipo [$\geq$ e =] x propriedade [degenerada]
f64	3	2	= $\geq$	MIN	OTIMA	4	SIM	SIM	1 0	Tipo [$\geq$ e =] x propriedade [degenerada+múltiplas]
f65	2	2	= $\geq$	MIN	INVIABEL	—	—	—	—	Tipo [$\geq$ e =] x propriedade [inviável]
f66	2	2	= $\geq$	MAX	ILIMITADA	—	—	—	—	Tipo [$\geq$ e =] x propriedade [ilimitada]
f71	2	3	$\leq$ = $\geq$	MIN	OTIMA	3.4	NAO	NAO	2 1	Tipo [$\leq$, $\geq$ e =] x propriedade [ótima única]
f72	3	3	$\leq$ = $\geq$	MAX	OTIMA	20	SIM	NAO	2 1	Tipo [$\leq$, $\geq$ e =] x propriedade [múltiplas]

caso	n	m	tipos	sentido	STATUS	Z	MULT	DEGEN	iterações	o que exercita
f73	3	4	<= = >=	MAX	OTIMA	18	NAO	SIM	2 2	Tipo [<=, >= e =] x propriedade [degenerada]
f74	3	3	<= = >=	MAX	OTIMA	4	SIM	SIM	1 1	Tipo [<=, >= e =] x propriedade [degenerada+múltiplas]
f75	3	3	<= = >=	MAX	INVIAVEL	—	—	—	—	Tipo [<=, >= e =] x propriedade [inviável]
f76	3	3	<= = >=	MAX	ILIMITADA	—	—	—	—	Tipo [<=, >= e =] x propriedade [ilimitada]

Grupo G — Escala crescente

Instâncias densas geradas com ótimo único conhecido de antemão (veja `gerador_grandes.py`). Servem para medir desempenho e para flagrar erros que só aparecem com muitas iterações: acúmulo de erro numérico, vazamento de memória, limite de iterações mal dimensionado.

caso	n	m			Z ótimo	iterações (F1+F2)	tempo da referência	arquivo
g01	5	6	0	0	2354.000000	0 5	0.00 s	incluído
g02	10	12	2	1	4238.000000	5 9	0.00 s	incluído
g03	20	25	4	2	27910.000000	11 18	0.00 s	incluído
g04	40	50	8	3	93688.000000	34 38	0.00 s	incluído
g05	80	100	15	5	458291.000000	90 84	0.01 s	incluído
g06	150	180	25	8	1602763.000000	129 176	0.03 s	incluído
g07	250	300	40	10	4234621.000000	273 328	0.09 s	incluído
g08	400	480	0	0	12157676.000000	0 1012	0.31 s	gerar com <code>gerador_grandes.py</code>
g09	600	720	60	15	26536326.000000	613 1304	1.19 s	gerar com <code>gerador_grandes.py</code>
g10	800	960	0	0	48155954.000000	0 2584	2.71 s	gerar com <code>gerador_grandes.py</code>
g11	1000	1200	100	20	71770571.000000	1585 2182	5.87 s	gerar com <code>gerador_grandes.py</code>
g12	1250	1500	0	0	117843774.000000	0 4821	11.54 s	gerar com <code>gerador_grandes.py</code>
g13	1600	1920	0	0	195438716.000001	0 6519	30.22 s	gerar com <code>gerador_grandes.py</code>

Tempos medidos com a implementação de referência em C (`gcc -O2`) num Xeon de 2,1 GHz. Uma implementação de aluno bem escrita costuma ficar na mesma ordem de grandeza; se o seu programa levar 5× mais que isso, procure cópias desnecessárias do tableau dentro do laço de pivoteamento.

Os casos `g08` a `g13` **não vêm prontos** (juntos passam de 20 MB). Gere-os com:

```
python3 gerador_grandes.py          # gera todos os que faltam
python3 gerador_grandes.py g13      # gera apenas um
```

g13 é o teto proposital da bateria: ~30 segundos na implementação de referência. Não é um caso de aprovação/reprovação — é o limite em que erros de desempenho e de memória ficam visíveis. Se ele terminar com o `Z` certo, seu programa está sólido.

Política de comparação

O script `verificar.py` compara a saída do seu programa com os arquivos de `esperado/` assim:

item	grupos A-F	grupo G
STATUS	exato — falha se diferir	exato — falha se diferir
Z	tolerância <code>1e-4</code> (relativa)	tolerância <code>1e-4</code> (relativa)
viabilidade de <code>X</code>	verificada nas restrições originais	verificada nas restrições originais
<code>c · X</code> bate com o <code>Z</code> impresso	sim	sim
<code>X</code> igual ao gabarito	falha (com <code>--estrito</code>) / aviso	aviso
MULTIPLAS , DEGENERADA	falha se diferir	aviso
ITERACOES	aviso (falha com <code>--estrito</code>)	apenas informativo
ALTERNATIVA	presença exigida; valor validado (viável e com mesmo <code>Z</code>)	idem

A frouxidão no grupo G é proposital: em instâncias grandes, diferenças de arredondamento podem levar duas implementações **igualmente corretas** a caminhos de pivô distintos e a vértices ótimos alternativos. O valor de `Z` e a viabilidade de `X`, esses sim, não têm desculpa para divergir.

Erro numérico: o que é aceitável

Um exemplo concreto está na própria tabela do grupo G: o valor certificado de `g13` é o inteiro `195438716`, e a implementação de referência imprime `195438716.000001`. Esse micrômetro de diferença, depois de 6519 pivoteamentos, é o comportamento **esperado** de aritmética `double` — e é por isso que o verificador compara `Z` por tolerância, nunca por igualdade exata.

Nos casos pequenos, sua saída deve bater dígito a dígito. Nos casos do grupo G, o acúmulo de milhares de pivoteamentos em `double` produz erro da ordem de `1e-9` a `1e-6` no valor de `Z` — por isso a tolerância relativa de `1e-4`. Se o seu `Z` divergir na terceira casa ou mais, o problema **não** é arredondamento: procure comparações com `==`, tolerâncias trocadas de sinal ou o teste da razão aceitando pivô muito próximo de zero.